

Telecom Signal Stream

Data Forge



A large orange shape on the left side of the slide, consisting of a rectangle with a quarter-circle cutout at the top right corner.

Team Members

- Ijaz Ahmad S
- Kavya Bhatta S
- Rakesh Kumar
- Sahana J Upadhyaya



Problem Statement

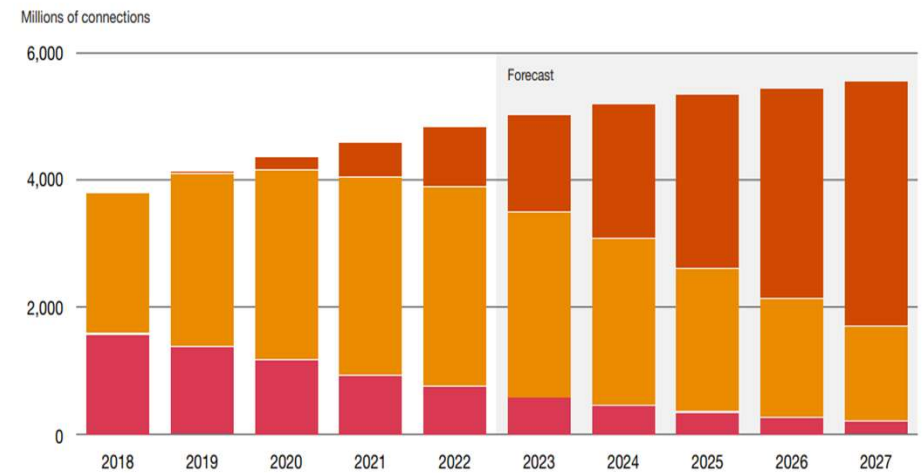
- Telecom companies generate huge volumes of mobile-signal data every day.
- Delayed or manual analysis slows down issue detection and response.
- Need a real-time solution to monitor and analyse network health instantly.

Objective

- Build a real-time analytics pipeline using AWS Kinesis, Glue, Athena, and CloudWatch.
- Enable continuous tracking of key metrics like signal strength, GPS precision, and network status.

Use Case

- Simulate mobile log data (CSV) as live stream via Kinesis.
- Perform ETL and cleaning using AWS Glue.
- Store curated data in S3 and query insights with Athena.
- Key Metrics:
 - Avg. Signal Strength per Region
 - GPS Accuracy per Postal Area and Operator
 - Network activity (Still, Tilting, Unknown, On_Foot, In_Vehicle)



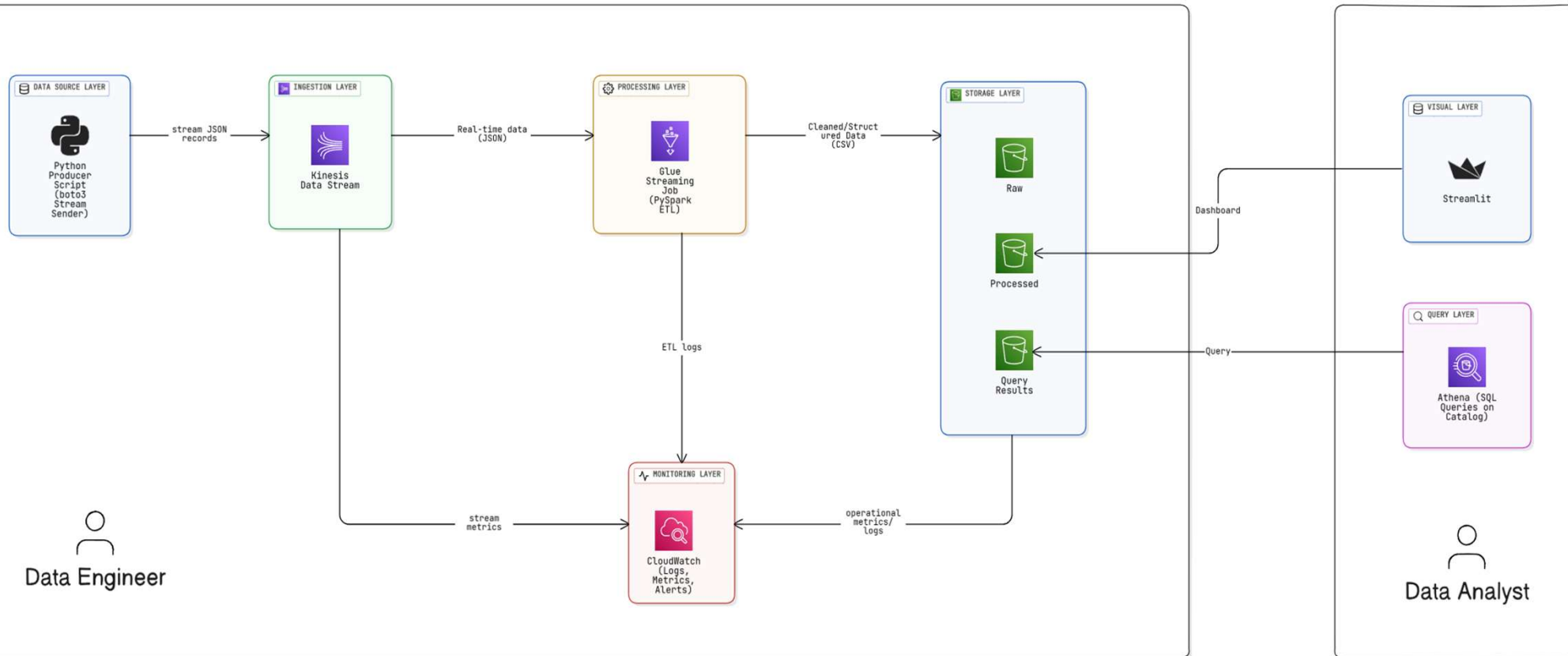
Note: 2018-2022 are actual numbers.
Source: PwC's Global Telecom Outlook 2023-2027, Omdia

Input Data Overview

- Schema & Sample
 - Dataset : mobile_logs
 - Format – csv
 - Streaming Data

hour	lat	long	signal	network	operator	status	description	speed	satellites	precision	provider	activity
00:36:07.000	9.7028	-16.9857	15	DTAC	JAZZTEL		0 STATE_IN_SERVICE	35.6	5.6	144.1	fused	STILL
00:36:08.000	3.68379	-2.76099	22	movistar	DTAC		0 STATE_IN_SERVICE	128.9	6.3	14.6	fused	STILL
00:36:09.000	28.02161	-3.30627	16	orange	RACC		0 STATE_IN_SERVICE	134.9	2.6	126.4	fused	UNKNOWN
00:36:10.000	16.51063	-9.35435	6	orange	vodafone ES		0 STATE_IN_SERVICE	45.3	11	139.9	fused	UNKNOWN
00:36:11.000	16.803	-12.9745	6	vodafone	Orange		0 STATE_IN_SERVICE	123	14.7	28	gps	IN_VEHICLE

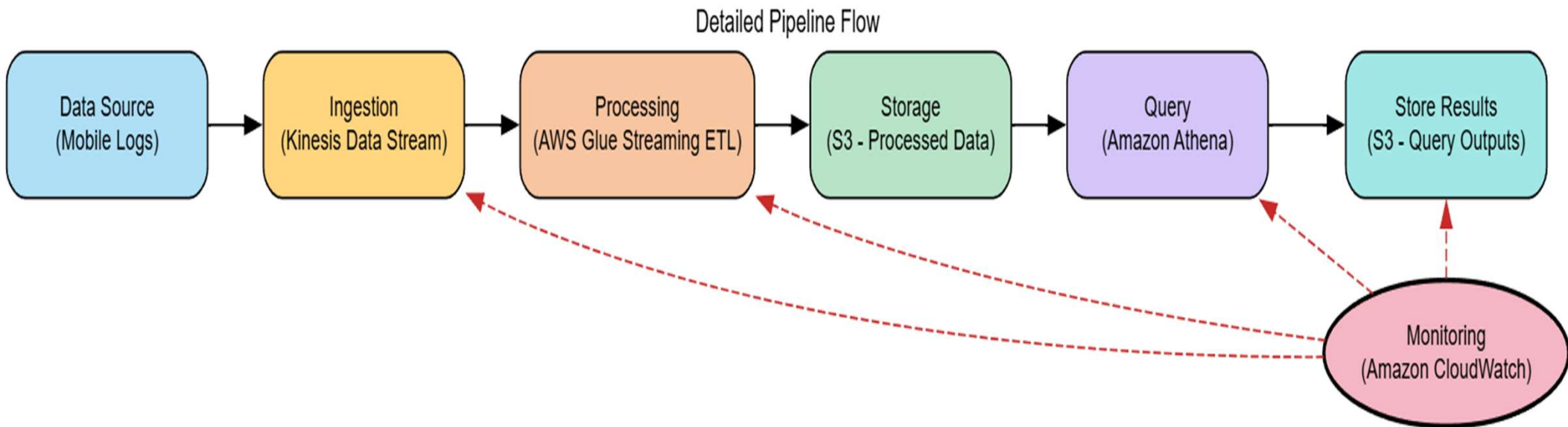
High-Level Architecture



Tools & Services Used:

Service	Purpose
Amazon S3	Central data lake (stores raw, processed, scripts, checkpoints, and query outputs).
Amazon Kinesis Data Stream	Real-time ingestion of mobile log data (acts as the streaming data source).
AWS Glue	Performs ETL processing (Spark Streaming job reads from Kinesis, writes to S3 in Parquet).
AWS IAM	Provides roles and permissions for Glue, Kinesis, and S3 access control.
Amazon Athena	Serverless query engine to analyze processed data stored in S3 using SQL.
Amazon CloudWatch	Monitors Glue job execution and logs for troubleshooting.

Detailed Pipeline Flow



Data Ingestion

- **Python Data Ingestion Script**

- Simulates real-time telecom data from CSV.
- Converts each row to JSON and sends it to Kinesis Data Stream.

```
import csv, json, time, boto3

STREAM_NAME = "kinesis-mobile-log-stream"
CSV_PATH = "mobile-logs.csv" # user's dataset path
SLEEP = 0.05

kinesis = boto3.client("kinesis", region_name="us-east-1")

with open(CSV_PATH, 'r') as f:
    reader = csv.DictReader(f)
    for i, row in enumerate(reader):
        data = json.dumps(row)
        kinesis.put_record(StreamName=STREAM_NAME, Data=data.encode('utf-8'), PartitionKey=str(i%10))
        if i % 50 == 0:
            print("sent", i)
            time.sleep(SLEEP)
```

```
sent 0
sent 50
sent 100
sent 150
sent 200
sent 250
sent 300
sent 350
```


S3 Bucket Setup

```
try:
    # 1. Create the S3 bucket
    s3_client.create_bucket(Bucket=bucket_name)
    print(f"Bucket '{bucket_name}' created.")
except ClientError as e:
    print(f"Error creating bucket: {e}")
```

```
folders = ['raw/', 'processed/', 'scripts/', 'checkpoint/', 'query-results/']

for folder in folders:
    try:
        s3_client.put_object(Bucket=bucket_name, Key=folder)
        print(f"Folder '{folder}' created.")
    except ClientError as e:
        print(f"Error creating folder '{folder}': {e}")
```

The screenshot shows the Amazon S3 console interface. The breadcrumb navigation at the top reads "Amazon S3 > Buckets > data-forge-bkt1". On the left sidebar, under "Amazon S3", the "General purpose buckets" section is expanded, showing a list of bucket types including Directory buckets, Table buckets, Vector buckets, Access Grants, Access Points (General Purpose Buckets, FSx file systems), Access Points (Directory Buckets), Object Lambda Access Points, Multi-Region Access Points, Batch Operations, and IAM Access Analyzer for S3.

The main content area is titled "Objects (5)". It features a toolbar with buttons for "Copy S3 URI", "Copy URL", "Download", "Open", "Delete", "Actions", and "Create folder". Below the toolbar is an "Upload" button and a text box for "Find objects by prefix". A descriptive paragraph explains that objects are fundamental entities in Amazon S3 and provides a link to "Amazon S3 inventory" for listing objects and another link for "permissions".

Below the text is a table listing the five folders in the bucket:

☐	Name	Type	Last modified	Size	Storage class
☐	checkpoint/	Folder	-	-	-
☐	processed/	Folder	-	-	-
☐	query-results/	Folder	-	-	-
☐	raw/	Folder	-	-	-
☐	scripts/	Folder	-	-	-

Kinesis Data Stream

Serverless Amazon Web Services (AWS) service for processing and analyzing real-time streaming data at scale.

kinesis-mobile-log-stream [Info](#) [Delete](#)

Data stream summary

Status ✔ Active	Data retention period 1 day	ARN arn:aws:kinesis:us-east-1:801341413974:stream/kinesis-mobile-log-stream	Creation time October 31, 2025 at 11:48 GMT+5:30
Capacity mode Provisioned	Maximum record size 1024 KiB		

< Applications Monitoring **Configuration** Enhanced fan-out (0) Data viewer Data analytics - new >

Data stream capacity [Info](#) [Edit capacity mode](#) [Edit provisioned shards](#)

Capacity mode Provisioned	Provisioned shards 1	Write capacity Maximum 1 MiB/second 1,000 records/second	Read capacity Maximum 2 MiB/second
------------------------------	-------------------------	---	--

```
try:
    response = kinesis_client.create_stream(
        StreamName=stream_name,
        ShardCount=shard_count
    )
    print(f"Creating Kinesis stream '{stream_name}' with {shard_count} shard(s)...")
except ClientError as e:
    print(f"Error creating stream: {e}")
    exit(1)
```

AWS Glue Job

```
try:
    response = glue.create_job(
        Name=job_name,
        Role=role_name,
        ExecutionProperty={
            'MaxConcurrentRuns': 1
        },
        Command={
            'Name': 'glueetl',
            'ScriptLocation': script_s3_path,
            'PythonVersion': '3'
        },
        DefaultArguments={
            '--extra-jars': dependent_jars
        },
        GlueVersion='5.0',
        Description='Streaming Glue job for DataForge project',
        Tags={
            'Project': 'DataForge',
            'Type': 'StreamingJob'
        }
    )
```

The screenshot shows the AWS Glue console interface. On the left is a navigation menu with options like 'Getting started', 'ETL jobs', 'Visual ETL', 'Notebooks', 'Job run monitoring', 'Data Catalog tables', 'Data connections', 'Workflows (orchestration)', 'Zero-ETL integrations', and 'Data Catalog'. The main panel displays the configuration for a job named 'DataForgeGlueJob1', which was last modified on 05/11/2025 at 07:06:29. The 'Job details' tab is active, showing three dropdown menus: 'Glue version' set to 'Glue 5.0 - Supports spark 3.5, Scala 2, Python 3', 'Language' set to 'Python 3', and 'Worker type' set to 'G 1X (4vCPU and 16GB RAM)'. At the top right of the main panel are buttons for 'Actions', 'Save', and 'Run'.

AWS Glue Streaming ETL Job (Status)

DataForgeGlueJob1

Last modified on 05/11/2025, 07:06:29

ActionsSaveRun

Script

Job details

Runs

Data quality

Schedules

Version Control

Job runs (1/1) Info

Last updated (UTC)
November 5, 2025 at 01:44:57

View details

Stop job run

Troubleshoot with AI

Table ViewCard View

Filter job runs by property

Run status

Retries

Start time (Local)

End time (Local)

Duration

Capacity (DP...

Worker type

Glue version

Running

0

11/05/2025 07:07:28

-

7 m 21 s

10 DPU

G.1X

5.0

Run details

Input arguments (1)

Logs

Run insights

Metrics

Troubleshooting analysis - preview

Spark UI

Stop job run

INFO

2025-11-05T01:40:44.973

171319

org.apache.spark.scheduler.TaskSetManager

[dispatcher-CoarseGrainedScheduler]

60

Starting task 23.0 in stage 9.0

INFO

2025-11-05T01:40:44.973

171319

org.apache.spark.scheduler.TaskSetManager

[dispatcher-CoarseGrainedScheduler]

60

Starting task 25.0 in stage 9.0

INFO

2025-11-05T01:40:44.974

171320

org.apache.spark.scheduler.TaskSetManager

[dispatcher-CoarseGrainedScheduler]

60

Starting task 26.0 in stage 9.0

INFO

2025-11-05T01:40:44.976

171322

org.apache.spark.scheduler.TaskSetManager

[dispatcher-CoarseGrainedScheduler]

60

Starting task 27.0 in stage 9.0

INFO

2025-11-05T01:40:44.977

171323

org.apache.spark.scheduler.TaskSetManager

[dispatcher-CoarseGrainedScheduler]

60

Starting task 28.0 in stage 9.0

INFO

2025-11-05T01:40:44.977

171323

org.apache.spark.scheduler.TaskSetManager

[dispatcher-CoarseGrainedScheduler]

60

Starting task 29.0 in stage 9.0

INFO

2025-11-05T01:40:44.958

171304

org.apache.spark.scheduler.TaskSetManager

[dispatcher-CoarseGrainedScheduler]

60

Starting task 7.0 in stage 9.0

INFO

2025-11-05T01:40:44.959

171305

org.apache.spark.scheduler.TaskSetManager

[dispatcher-CoarseGrainedScheduler]

60

Starting task 8.0 in stage 9.0

KPI's

- Average signal by operator

#	▼	operator	▼	Avg_Signal
1		Eroski Movil		13.214285714285714
2		simyo		13.636363636363637
3		RACC		14.941176470588236
4		vodafone ES		20.17241379310345
5		Tuenti		11.038461538461538
6		GETESA		15.26086956521739
7		DTAC		14.263157894736842
8		Movistar		15.074074074074074
9		JAZZTEL		14.541666666666666
10		Orange		16.0
11		VIVO		14.805555555555555
12		YOIGO		13.625

- Average signal by postal code

#	▼	postal_code	▼	Avg_Signal
1		15673		21.0
2		15680		9.0
3		15707		21.0
4		15722		8.0
5		15736		27.0
6		15756		18.0
7		15792		22.0
8		15821		22.0
9		15955		21.0
10		16216		17.0

KPI's

- Average signal by operator and postal code

#	▼	operator	▼	postal_code	▼	Avg_Signal	▼
1		RACC		16328.		10.0	
2		GETESA		16445.		20.0	
3		VIVO		16482.		3.0	
4		VIVO		16636.		27.0	
5		RACC		16648.		28.0	
6		DTAC		16809.		24.0	
7		VIVO		16850.		30.0	
8		Eroski Movil		16940.		10.0	
9		Tuenti		17005.		8.0	
10		Movistar		17241.		17.0	

- Average GPS precision by provider

#	▼	operator	▼	Avg_precision
1		vodafone ES		76.75882352941177
2		simyo		68.58611111111112
3		RACC		89.70333333333335
4		Eroski Movil		72.59411764705882
5		Tuenti		93.69615384615386
6		JAZZTEL		72.72142857142856
7		Movistar		75.685
8		Orange		67.90416666666665
9		GETESA		65.72
10		VIVO		88.56923076923077

KPI's

Activity Count

#	▼	activity	▼	Activity_Count
1		ON_BICYCLE		119
2		UNKNOWN		126
3		TILTING		101
4		IN_VEHICLE		101
5		STILL		114
6		ON_FOOT		118

Business Output

- Real-time data availability : Incoming mobile log data instantly processed and stored in S3.
- Clean & structured datasets : JSON stream transformed into timestamped, analytics-ready CSV files.
- Faster insights : Teams can query live data using Amazon Athena without manual ETL steps.
- Scalable foundation : The pipeline can easily integrate new data sources or analytics dashboards (QuickSight, Power BI).



Key Learnings

1. AWS Data Pipeline Design : Learned how to build a real-time data pipeline connecting Kinesis Data Stream → AWS Glue → S3 → Athena for end-to-end data processing.
2. Spark Structured Streaming : Gained hands-on experience with reading continuous streams and writing results.
3. Schema Handling & Data Transformation : Practiced defining JSON schemas, parsing dynamic data, and transforming raw data into structured CSV output.
4. Offline Testing & Local Simulation : Developed a local test setup using Docker + Spark to simulate streaming with JSON files before deploying to AWS.
5. AWS Cloud Integration : Explored Glue Catalog, Athena, and S3 as a seamless ecosystem for data storage, querying, and analytics.

Future Scope



Real-Time Analytics Dashboard

Integrate with Amazon QuickSight or Grafana for live visualization of streaming data.



Data Quality & Validation Layer

Add AWS Lambda or Glue triggers to validate and cleanse data before storage.



Scalable Multi-Stream Integration

Extend the pipeline to handle multiple Kinesis streams for different data sources or applications.

Thank you
